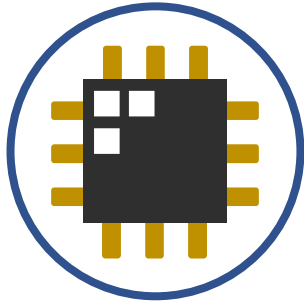




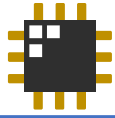
EEE 143: Switching Theory and Digital Logic Design

Lecture 4 – Karnaugh Maps (K-maps)

1

This lecture introduces something new and helpful technique, especially coming from Boolean manipulation. The biggest challenge in Boolean algebra is simplifying expressions using only its properties. Often, it's difficult to determine whether you've reached the simplest form or if further simplification is possible. This uncertainty can make Boolean algebra frustrating, you simplify without a clear target, relying on intuition rather than a definite stopping point.

Now, with this lecture, we will be showing a technique that will allow us to determine the simplest expression starting from a truth table. As long as you are doing the technique correctly, you will arrive at the simplest expression.



Minimizing Boolean Expressions

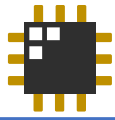
- Recall: Implementing logic circuits comes at a cost. **Cost** could mean **power consumption**, **area** occupied by the circuit, **delay**, etc.
 - For example, cost may be assigned to a logic circuit based on how many logic gates are being used:
 - $E = [(AC \cdot W'X') + (AC' \cdot WX)] + [(AC \cdot Y'Z) + (AC' \cdot YZ')]$
 - **Cost:** 3 two-input OR gates, 12 two-input AND gates, 5 inverters
- Generally, reducing the number of literals (variable or its complement) in a Boolean expression reduces cost.
 - So far, we have done this through Boolean algebra

2

Quick recall, everything we discussed in the last meeting is motivated by cost. Cost can take different forms, such as power consumption, area, or delay, but ultimately, all these factors are proportional to the complexity of the circuit. The more logic gates required to implement a system, the higher the cost.

For example, given the Boolean expression above, we need three two-input OR gates, twelve two-input AND gates, and five inverters. Another way to think about cost is in terms of the number of variables. When we aim to simplify an expression, our goal is to reduce the number of terms. This, in turn, decreases the number of logic gates needed for implementation. By minimizing literals (or variables), we effectively reduce the overall circuit cost.

So far, we have tackled only one technique --- minimization through pure Boolean algebra. We highlighted different theorems that we can exploit to allow you to further simplify the expression.



Minimizing Boolean Expressions

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$A'B'C'$

$A'B'C$

$AB'C'$

$AB'C$

$$Q = A'B'C' + A'B'C + AB'C' + AB'C$$

3

Now, starting from the truth table, we'll end up with a standard sum of products. So again to recall, if we are trying to write an expression from a truth table, we look at rows where the output is 1 and then we just name each row, depending on what the input combination is.

So, for the first column, A is 0, B is 0, and C is 0, hence, $A'B'C'$

For the second column, A is 0, B is 0, and C is 1, hence, $A'B'C$

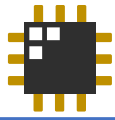
For the fifth column, A is 1, B is 0, and C is 0, hence, $AB'C'$

Lastly, for the sixth column, A is 1, B is 0, and C is 1, hence, $AB'C$

Then after generating the expression per row, we OR them together, ending up with an expression:

$$Q = A'B'C' + A'B'C + AB'C' + AB'C$$

However again, this expression is just the standard sum of products, so it is possible to have a more simplified version.



Minimizing Boolean Expressions

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$A'B'C'$

$A'B'C$

$AB'C'$

$AB'C$

$$Q = A'B'C' + A'B'C + AB'C' + AB'C$$

$$Q = A'B'(C' + C) + AB'(C' + C)$$

$$Q = A'B' + AB'$$

$$Q = B'(A' + A)$$

$$Q = B'$$

Let's take a closer look at what's *really* happening!

Now, let's simplify the standard sum of products expression from the previous slide.

For $Q = A'B'(C'+C) + AB'(C'+C)$: We apply the distributive property in reverse, factoring out the common terms.

For $Q = A'B' + AB'$: Here, we use the Complement Theorem to simplify $C'+C = 1$, and then apply the Identity Property to remove the 1.

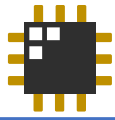
For $Q = B'(A'+A)$: We factor out B' using the distributive property.

For $Q = B'$: Here, we apply the Complement Theorem to simplify $A'+A = 1$, and once again use the Identity Property to remove the 1.

What we've done here is pure Boolean manipulation. Every step follows Boolean theorems to produce an equivalent expression. If we construct a truth table for the simplified expression and compare it to the original, they will be identical.

For example, looking at $Q = B'$, we see that output is 1 only when $B = 0$ ---just as in the original expression. This confirms that our simplification preserves the logic of the

circuit.



Minimizing Boolean Expressions

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$A'B'(C'+C)$

$AB'C'$

$AB'C$

$$Q = \underline{A'B'C'} + \underline{A'B'C} + \underline{AB'C'} + \underline{AB'C}$$

$$Q = \underline{A'B'(C' + C)} + AB'(C' + C)$$

Looking at these two rows...

A=0 and B=0 are common
C can be either 0 or 1

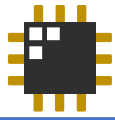
5

Now, let's take a closer look at the table and analyze what happens at each step of the minimization process.

Consider the term $A'B'(C'+C)$ that we obtained when factoring out $A'B'$. Looking back at the truth table, we can see that this expression corresponds to the first two rows. Essentially, we are examining these two rows together, and they are both described by $A'B'(C'+C)$.

What can we notice about these two rows? In both rows, $A = 0$ and $B = 0$, but C is **either 0 or 1**. We can see that it really doesn't matter what C is, output is 1 as long as A and B are both 0.

Thus, the expression simplifies to $A'B'$.



Minimizing Boolean Expressions

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$A'B'(C'+C)$

$AB'(C'+C)$

$$Q = \underline{A'B'C'} + \underline{A'B'C} + \underline{AB'C'} + \underline{AB'C}$$

$$Q = A'B'(C' + C) + \underline{AB'(C' + C)}$$

Looking at these two rows...

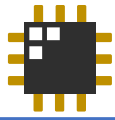
A=1 and B=0 are common
C can be either 0 or 1

6

Again, we are looking at two rows that are described by $AB'(C'+C)$, which follows the same pattern as the previous slide.

However, this time, $A = 1$ and $B = 0$ and just like before, the value of C doesn't matter.

Since the output is 1 as long as A is 1 and B is 0, we can conclude that the expression simplifies to AB' .



Minimizing Boolean Expressions

A	B	C	Q	
0	0	0	1	A'B'
0	0	1	1	
0	1	0	0	
0	1	1	0	
1	0	0	1	AB'
1	0	1	1	
1	1	0	0	
1	1	1	0	

$$Q = \underline{A'B'C'} + \underline{A'B'C} + \underline{AB'C'} + \underline{AB'C}$$

$$Q = A'B'(C' + C) + AB'(C' + C)$$

$$Q = \underline{A'B'} + \underline{AB'}$$

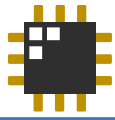
7

Now, let's look at the four highlighted rows at the same time, considering that we don't care at the values of C, what are things that are common on those four rows? The outputs are all one, when B is 0 and when A is either 1 or 0. Hence, we can say that we don't care what A is, as long as B is 0. Considering this, we can say that the expression is indeed $Q = B'$

Now, let's look at the four highlighted rows together. Since we don't care about the values of C, what do these rows have in common?

In all four rows, the output is 1 when $B = 0$, regardless of whether $A = 1$ or $A = 0$. This means that as long as $B = 0$, the output will always be 1.

Thus, we can conclude that the expression simplifies to $Q = B'$.



Minimizing Boolean Expressions

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$A'B'$

$B'(A'+A)$

AB'

$$Q = \underline{A'B'C'} + \underline{A'B'C} + \underline{AB'C'} + \underline{AB'C}$$

$$Q = A'B'(C' + C) + AB'(C' + C)$$

$$Q = \underline{A'B'} + \underline{AB'}$$

$$Q = B'(\underline{A'} + \underline{A})$$

Looking at these two groups...

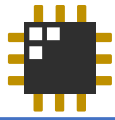
Ignoring C for now

B=0 is common

A can be either 0 or 1

Following the same logic as the previous two slides and temporarily ignoring **C**, let's examine these four rows together and identify their common pattern.

We can see that **B = 0** in all four rows, while **A** can be either **0** or **1**. Since the output remains **1** whenever **B = 0**, regardless what **A** is, we can conclude that the expression simplifies to **B'**.



Minimizing Boolean Expressions

A	B	C	Q	
0	0	0	1	$A'B'C'$
0	0	1	1	$A'B'C$
0	1	0	0	
0	1	1	0	
1	0	0	1	$AB'C'$
1	0	1	1	$AB'C$
1	1	0	0	
1	1	1	0	

$$Q = \underline{A'B'C'} + \underline{A'B'C} + \underline{AB'C'} + \underline{AB'C}$$

$$Q = A'B'(C' + C) + AB'(C' + C)$$

$$Q = A'B' + AB'$$

$$Q = B'(A' + A)$$

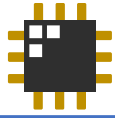
$$Q = B'$$

Groupings in the truth table reflect what's happening in the algebra!

9

Earlier, we analyzed B' by completing the truth table step by step. This time, however, we grouped terms directly within the truth table itself. By examining the truth table, we derived expressions without relying solely on Boolean algebra.

For now, we are translating these observations into their algebraic form. However, once you become more familiar with recognizing patterns, you can eventually skip the Boolean algebra steps altogether. Simply by looking at the truth table and identifying patterns, you'll be able to write the simplified expression immediately.



Identifying possible 'groupings'

- What if...

- Instead of doing pure Boolean algebra/manipulation, we look for **patterns** within the truth table itself?
 - We check for rows that both have an output of 1 and input combinations differ by 1 bit
 - We then 'name' the group by the common terms between the rows instead

A	B	C	Q
0	0	0	1
0	0	1	1

$A'B'(C'+C)$

Looking at these two rows...

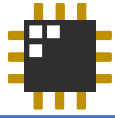
A=0 and B=0 are common
C can be either 0 or 1

The equivalent Boolean expression for this grouping: **$A'B'$**

10

As previously mentioned, instead of relying solely on Boolean manipulation, we can analyze the truth table directly. By identifying patterns within the table, we can derive the simplified expression much easier.

For example, when identifying rows where the output is **1**, we notice that their input combinations differ by only **one bit**. Take the first two rows: in both cases, **A = 0** and **B = 0**, while only **C** changes. That's why we concluded that these two rows are represented by **$A'B'$** —because as long as **A = 0** and **B = 0**, the output is always **1**, regardless of **C**.



Identifying possible 'groupings'

A	B	C	Q	
0	0	0	1	$A'B'$
0	0	1	1	
0	1	0	0	$B'(A'+A)$
0	1	1	0	
1	0	0	1	AB'
1	0	1	1	
1	1	0	0	
1	1	1	0	

These two groups ($A'B'$ and AB') also have an output of 1 and input combinations differing by 1 bit

Looking at these two groups...

$B=0$ is common

A can be either 0 or 1

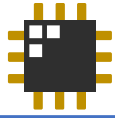
The equivalent Boolean expression for this grouping: B'

11

Following the same logic, let's examine these four rows while ignoring C. What do they all have in common?

We notice that the input combinations differ by only **one bit**, with A being the only changing variable, while $B = 0$ in all four rows.

Since the output remains 1 regardless of A, we can simplify the expression to B' .



Identifying possible 'groupings'

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

B'

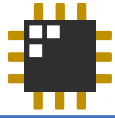
However, this large group of rows is *spaced apart* when looking at the truth table...

- Hard to find 'patterns' this way!

Let's rearrange the truth table to see these 'patterns' better!

12

Now that we understand how to approach this process, the only challenge left is spotting these patterns in a truth table. Instead of identifying them step by step, why not analyze all four rows at once? But how can we make that easier? Let's rearrange the truth table to make patterns more visually apparent.



Rearranging the truth table

- Construct a grid with as many cells as there are rows for the given number of input variables
 - As you move from one cell to another, one variable changes each time a boundary is cut or crossed (**1-bit change**, also called logically adjacent)
 - For two bits, for example: the pattern is **00** -> **01** -> **11** -> **10**
 - 1-bit change every time we transition!
 - Each cell in the grid maps a row in the truth table

13

What we've observed so far is that when grouping rows, we want each pair to differ by only **one bit**—meaning that between two grouped rows, only **C** changes while the other variables remain the same.

Now, let's try rearranging the order of the truth table so that every adjacent cell follows this **one-bit change** pattern. The idea is to organize the truth table elements into a **grid**, ensuring that adjacent cells differ by only a **single bit**.

For example, if we have two bits, a valid **one-bit change sequence** would be:

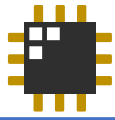
00 → 01 → 11 → 10

00 → 01 → Only the first bit changes

01 → 11 → Only the second bit changes

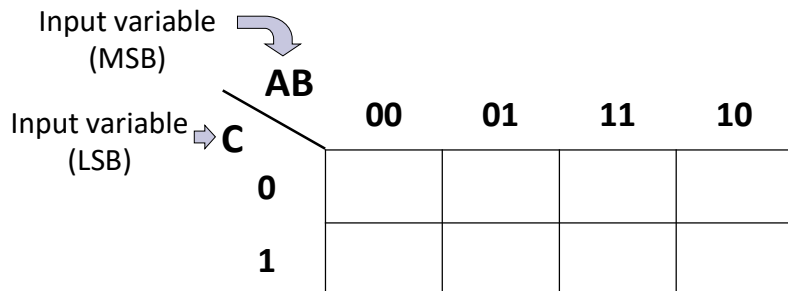
11 → 10 → Only the first bit changes again

By structuring the table this way, we make it easier to identify patterns and simplify expressions efficiently.



Rearranging the truth table

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0



Note:

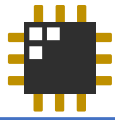
MSB = Most Significant Bit

LSB = Least Significant Bit

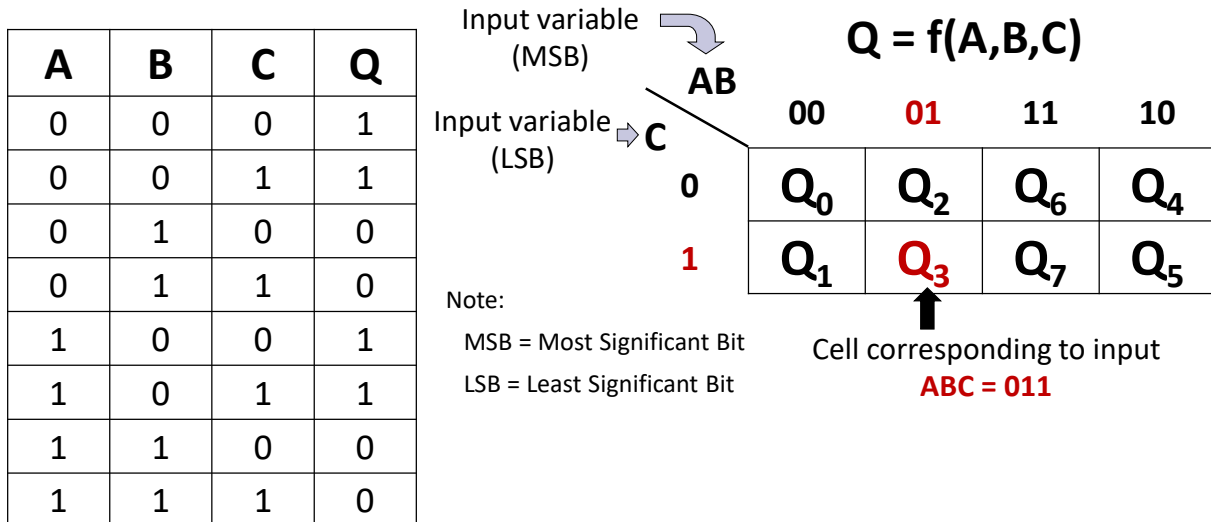
14

On the left, we have the original truth table, and on the right, we see the newly structured grid. The grid consists of **four columns**, representing the four possible combinations of **AB**, arranged according to the **one-bit change pattern** we discussed earlier: **00 $\rightarrow$ 01 $\rightarrow$ 11 $\rightarrow$ 10**. For **C**, we only have two possible values: **0 and 1**.

It's also worth noting that we can swap the positions of the **MSB** and **LSB**, but for consistency, we'll follow the convention shown here in the upcoming slides.



Rearranging the truth table



15

Now, let's take a closer look at how each cell corresponds to a specific combination of inputs.

Each cell represents a unique **minterm**, which is determined by the values of the input variables. If we have three variables **A**, **B**, and **C**, we can map an input combination to its corresponding cell using the following steps:

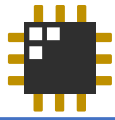
1. **Find the column** labeled with the **AB** value.
2. **Find the row** labeled with the **C** value.
3. **Locate the intersection**, which is the cell corresponding to that input combination.

For example, let's determine the cell for **A = 0, B = 1, and C = 1 (011 in binary)**.

- First, find the column labeled **01** for **AB**.
- Then, find the row labeled **1** for **C**.
- The intersection of this column and row gives us the cell for **011**, which corresponds to **Q_3** .

Additionally, take note of the numbering of each cell. The label of a cell, such as **011** $\rightarrow$ **3**, **000** $\rightarrow$ **0**, **110** $\rightarrow$ **6**, and **100** $\rightarrow$ **4**, follows the **decimal equivalent of the binary input combination**. This numbering helps in quickly identifying the corresponding

minterm for each cell in the K-Map.



Rearranging the truth table

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Input variable (MSB) $\rightarrow$ AB

Input variable (LSB) $\rightarrow$ C

$Q = f(A,B,C)$

	00	01	11	10
0	Q_0	Q_2	Q_6	Q_4
1	Q_1	Q_3	Q_7	Q_5

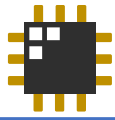
Note:
MSB = Most Significant Bit
LSB = Least Significant Bit

Note the ordering for this section!

Also, note how the input values change by only **one bit** as we move from one cell to another...

16

Take note of the ordering in the highlighted section. You might notice that the cell arrangement seems scrambled compared to standard binary numbering. However, this follows a specific pattern to ensure that each adjacent cell differs by only **one bit**. This structure is important for simplifying Boolean expressions, as it reduces the risk of overlooking valid groupings and makes it easier to identify patterns for minimization.



Rearranging the truth table

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Input variable (MSB) $\curvearrowright$ AB

Input variable (LSB) $\rightarrow$ C

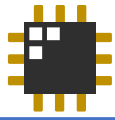
$Q = f(A,B,C)$

	00	01	11	10
0	Q_0	Q_2	Q_6	Q_4
1	Q_1	Q_3	Q_7	Q_5

At this point, we simply put the corresponding truth table outputs per cell

17

Now, that we know how to construct a grid given from the truth table, we will now put the corresponding truth table outputs per cell.



Rearranging the truth table

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

		AB			
C		00	01	11	10
0					
1					

**Write the corresponding outputs
per input combination!**

18

Now, let's write the corresponding outputs per input combination!

We can simply refer to the of the truth table and fill in the grid accordingly.

Again, note that:

Q0 | Q2 | Q6 | Q4

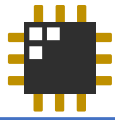
Q1 | Q3 | Q7 | Q5

Now, filling in the outputs from the truth table:

1 | 0 | 0 | 1

1 | 0 | 0 | 1

By organizing the truth table this way, we can easily identify patterns and groupings, leading to a simplified Boolean expression.



Rearranging the truth table

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

		AB			
		00	01	11	10
C	0	1	0	0	1
	1	1	0	0	1

19

Now, let's write the corresponding outputs per input combination!

We can simply refer to the of the truth table and fill in the grid accordingly.

Again, note that:

Q0 | Q2 | Q6 | Q4

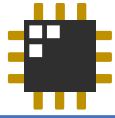
Q1 | Q3 | Q7 | Q5

Now, filling in the outputs from the truth table:

1 | 0 | 0 | 1

1 | 0 | 0 | 1

By organizing the truth table this way, we can easily identify patterns and groupings, leading to a simplified Boolean expression.



Rearranging the truth table

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

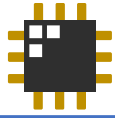
		AB			
		00	01	11	10
C	0	1	0	0	1
	1	1	0	0	1

This is what we call a Karnaugh Map (K-map)!

20

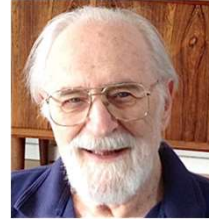
What we just created is called Karnaugh Map (K-map)!

Basically, Karnaugh Map (K-map) is a structured way of visualizing a truth table to simplify Boolean expressions. The values placed inside the K-map cells represent the output of the function for specific input conditions. This layout/grid makes it easier to identify patterns and minimize expressions efficiently.



Karnaugh Maps (K-maps)

- Invented in 1953 by **Maurice Karnaugh**, a telecommunications engineer at Bell Labs
- Graphical method for simplifying Boolean equations
- Work well for problems with up to 4 variables
- Give important insights into manipulating Boolean equations



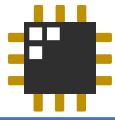
Maurice Karnaugh, 1924–. Graduated with a bachelor's degree in physics from the City College of New York in 1948 and earned a Ph.D. in physics from Yale in 1952. Worked at Bell Labs and IBM from 1952 to 1993 and as a computer science professor at the Polytechnic University of New York from 1980 to 1999.

Read more about him at
<https://www.ithistory.org/honor-roll/dr-maurice-karnaugh>

21

Maurice Karnaugh is an American physicist and engineer best known for developing the Karnaugh Map in 1953 while working on Bell Labs. His work significantly contributed to Boolean algebra simplification, providing a visual method to minimize logical expressions without the need for extensive algebraic manipulation.

His work has had a lasting impact on digital logic design, making the K-map a fundamental tool in electronics, particularly in minimizing logic gate usage in combinational circuits.



Example 1: Draw the K-map

$$F(A,B,C) = A'B'C' + A'BC' + AB'C' + A'BC$$

AB \ C		00	01	11	10
C	0				
	1				

22

Now, let's go through a practice example of mapping a Boolean function into a K-map.

Given the Boolean function:

$$F(A,B,C) = A'B'C' + A'BC' + AB'C' + A'BC$$

Let's convert each term into its binary representation:

$$A'B'C' \rightarrow 000$$

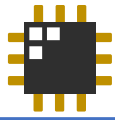
$$A'BC' \rightarrow 010$$

$$AB'C' \rightarrow 100$$

$$A'BC \rightarrow 011$$

Following the standard K-map arrangement, we assign 1 to the positions corresponding to the minterms(input combinations):

$$\begin{array}{c} 1 \mid 1 \mid 0 \mid 1 \\ 0 \mid 1 \mid 0 \mid 0 \end{array}$$



Example 1: Draw the K-map

$$F(A,B,C) = A'B'C' + A'BC' + AB'C' + A'BC$$

C \ AB	AB			
	00	01	11	10
0	1	1	0	1
1	0	1	0	0

23

Now, let's go through a practice example of mapping a Boolean function into a K-map.

Given the Boolean function:

$$F(A,B,C) = A'B'C' + A'BC' + AB'C' + A'BC$$

Let's convert each term into its binary representation:

$$A'B'C' \rightarrow 000$$

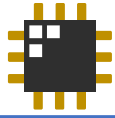
$$A'BC' \rightarrow 010$$

$$AB'C' \rightarrow 100$$

$$A'BC \rightarrow 011$$

Following the standard K-map arrangement, we assign 1 to the positions corresponding to the minterms(input combinations):

$$\begin{array}{c} 1 \mid 1 \mid 0 \mid 1 \\ 0 \mid 1 \mid 0 \mid 0 \end{array}$$



Example 2: Draw the K-map

$$F(A,B,C) = (A+B+C')(A+B'+C)(A'+B'+C')(A'+B+C)$$

AB \ C	00	01	11	10
0				
1				

24

Now, let's work through a slightly more confusing example. We are given the Boolean function:

$$F(A,B,C) = (A+B+C')(A+B'+C)(A'+B'+C')(A'+B+C)$$

This expression is in **Standard Product of Sums (SPOS) form**, meaning each term represents a **maxterm**—a case where the function outputs **0**.

To determine the corresponding maxterms, recall that:

- A **non-primed** variable (e.g., A) represents **0** in the binary form.
- A **primed** variable (e.g., A') represents **1** in the binary form.

Now, let's convert each sum term to its binary form:

$$(A+B+C') = 001$$

$$(A+B'+C) = 010$$

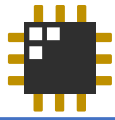
$$(A'+B'+C') = 111$$

$$(A'+B+C) = 100$$

Since these represent **maxterms**, their output in the truth table is **0**. The remaining

minterms will have an output of **1**. Hence, we will have:

1		0		1		0
0		1		0		1



Example 2: Draw the K-map

$$F(A,B,C) = (A+B+C')(A+B'+C)(A'+B'+C')(A'+B+C)$$

C \ AB	AB			
	00	01	11	10
0	1	0	1	0
1	0	1	0	1

25

Now, let's work through a slightly more confusing example. We are given the Boolean function:

$$F(A,B,C) = (A+B+C')(A+B'+C)(A'+B'+C')(A'+B+C)$$

This expression is in **Standard Product of Sums (SPOS) form**, meaning each term represents a **maxterm**—a case where the function outputs **0**.

To determine the corresponding maxterms, recall that:

- A **non-primed** variable (e.g., A) represents **0** in the binary form.
- A **primed** variable (e.g., A') represents **1** in the binary form.

Now, let's convert each sum term to its binary form:

$$(A+B+C') = 001$$

$$(A+B'+C) = 010$$

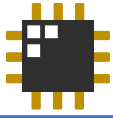
$$(A'+B'+C') = 111$$

$$(A'+B+C) = 100$$

Since these represent **maxterms**, their output in the truth table is **0**. The remaining

minterms will have an output of **1**. Hence, we will have:

1		0		1		0
0		1		0		1



4-variable K-map

CD \ AB				
	00	01	11	10
00				
01				
11				
10				

26

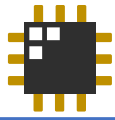
Now that we have four variables, we will have a total of $2^4 = 16$ possible input combinations.

For a 4-variable function, we split the variables into two groups:

- The first two variables AB determine the columns.
- The last two variables CD determine the rows.

This results in a 4x4 grid, where:

- The columns are labeled AB in a one-bit change order.
- The rows are labeled CD in a one-bit change order.



4-variable K-map

CD \ AB				
	00	01	11	10
00	Q_0	Q_4	Q_{12}	Q_8
01	Q_1	Q_5	Q_{13}	Q_9
11	Q_3	Q_7	Q_{15}	Q_{11}
10	Q_2	Q_6	Q_{14}	Q_{10}

Take care of the ordering!

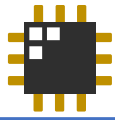
The ordering of the cells is a little 'weird' due to logical adjacency

(00→01→11→10)

which is not the typical binary order

27

Be mindful of the ordering in a 4-variable K-map, as it follows a one-bit change order, rather than standard binary order. Incorrect ordering can lead to mistakes in identifying adjacent groupings, potentially resulting errors when simplifying Boolean expressions.

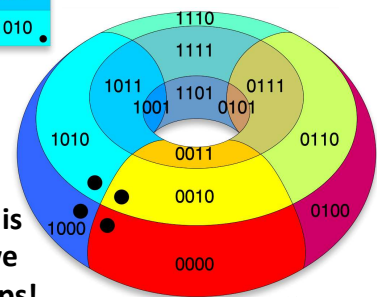


4-variable K-map

AB \ CD	00	01	11	10
00	Q_0	Q_4	Q_{12}	Q_8
01	Q_1	Q_5	Q_{13}	Q_9
11	Q_3	Q_7	Q_{15}	Q_{11}
10	Q_2	Q_6	Q_{14}	Q_{10}

Also, note how the first column/row is *logically adjacent* to the last column/row!

0000	0100	1100	1000
0001	0101	1101	1001
0011	0111	1111	1011
0010	0110	1110	1010



We'll leverage this property once we start making groups!

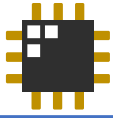
Image credit: "Karnaugh map" wikipedia.org

28

In a **4-variable K-map**, the rows and columns are arranged so that each cell differs from its neighbor by only **one bit**. This setup helps in easily identifying and grouping common terms for simplification.

Important note:

The **first and last columns/rows** are **logically adjacent**, even though they appear on opposite sides. Think of the K-map as a **loop** or a **donut**, where these edges wrap around and connect. This means that if the outputs in these cells are the same, they can be grouped together for simplification.



Example 3: Draw the K-map

$$F(A,B,C,D) = \sum m(0, 2, 3, 6, 7, 8, 10, 14, 15)$$

CD \ AB				
	00	01	11	10
00				
01				
11				
10				

29

Let's move to the next example. Given the function:

$$F(A,B,C,D) = \sum m(0,2,3,6,7,8,10,14,15)$$

Again, note that the given is a sum of minterms, meaning that the terms here outputs 1.

Now, consider the 4-variable K-map:

Q0 | Q4 | Q12 | Q8

Q1 | Q5 | Q13 | Q9

Q3 | Q7 | Q15 | Q11

Q2 | Q6 | Q14 | Q10

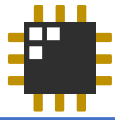
Each Q_n represents minterm n , and we write 1 in the cells corresponding to minterms in the given function. Hence, we'll have:

1 | 0 | 0 | 1

0 | 0 | 0 | 0

1 | 1 | 1 | 0

1 | 1 | 1 | 1



Example 3: Draw the K-map

$$F(A,B,C,D) = \sum m(0, 2, 3, 6, 7, 8, 10, 14, 15)$$

CD \ AB	AB			
	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	1	1	1	0
10	1	1	1	1

30

Let's move to the next example. Given the function:

$$F(A,B,C,D) = \sum m(0,2,3,6,7,8,10,14,15)$$

Again, note that the given is a sum of minterms, meaning that the terms here outputs 1.

Now, consider the 4-variable K-map:

Q0 | Q4 | Q12 | Q8

Q1 | Q5 | Q13 | Q9

Q3 | Q7 | Q15 | Q11

Q2 | Q6 | Q14 | Q10

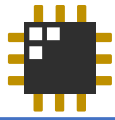
Each Q_n represents minterm n , and we write 1 in the cells corresponding to minterms in the given function. Hence, we'll have:

1 | 0 | 0 | 1

0 | 0 | 0 | 0

1 | 1 | 1 | 0

1 | 1 | 1 | 1



5-variable K-map

K-maps are usually drawn in 4-variable grids

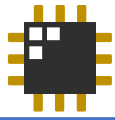
A = 0					A = 1						
BC						BC					
DE	00	01	11	10	DE	00	01	11	10		
00	Q_0	Q_4	Q_{12}	Q_8	00	Q_{16}	Q_{20}	Q_{28}	Q_{24}		
01	Q_1	Q_5	Q_{13}	Q_9	01	Q_{17}	Q_{21}	Q_{29}	Q_{25}		
11	Q_3	Q_7	Q_{15}	Q_{11}	11	Q_{19}	Q_{23}	Q_{31}	Q_{27}		
10	Q_2	Q_6	Q_{14}	Q_{10}	10	Q_{18}	Q_{22}	Q_{30}	Q_{26}		

A 5-variable K-map extends the concept of 4-variable K-maps by adding another dimension. Since each additional variable doubles the number of possible input combinations, a 5-variable K-map contains 32 cells (2^5).

The K-map is visualized as two separate 4-variable K-maps, one for when 1st variable A is 0 and another when A is 1.

Still, each of these 4-variable K-maps follows the one-bit change order.

The two K-maps themselves are adjacent, such as the corresponding cells between two maps (same row and column) differ only by A.



Example 4: Draw the K-map

$$F(A,B,C,D,E) = \prod M(4, 6, 12, 14, 20, 21, 22, 23)$$

A = 0					A = 1						
BC						BC					
DE	00	01	11	10	DE	00	01	11	10		
00					00						
01					01						
11					11						
10					10						

Let's move on to the next example. We have the function:

$$F(A,B,C,D,E) = \prod M(4, 6, 12, 14, 20, 21, 22, 23)$$

Again, recall that for product of maxterms, the output for the corresponding max terms are 0.

Now, take note of the 5-variable K-map:

A = 0

Q0 | Q4 | Q12 | Q8

Q1 | Q5 | Q13 | Q9

Q3 | Q7 | Q15 | Q11

Q2 | Q6 | Q14 | Q10

A = 1

Q16 | Q20 | Q28 | Q24

Q17 | Q21 | Q29 | Q25

Q19 | Q23 | Q31 | Q27

Q18 | Q22 | Q30 | Q26

We just fill in the outputs of 0 for the maxterms then place 1 on the unfilled cells:

A = 0

1 | 0 | 0 | 1

1 | 1 | 1 | 1

1 | 1 | 1 | 1

1 | 0 | 0 | 1

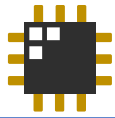
A = 1

1 | 0 | 1 | 1

1 | 0 | 1 | 1

1 | 0 | 1 | 1

1 | 0 | 1 | 1



Example 4: Draw the K-map

$$F(A,B,C,D,E) = \prod M(4, 6, 12, 14, 20, 21, 22, 23)$$

A = 0					A = 1						
BC						BC					
DE	00	01	11	10	DE	00	01	11	10		
00	1	0	0	1	00	1	0	1	1		
01	1	1	1	1	01	1	0	1	1		
11	1	1	1	1	11	1	0	1	1		
10	1	0	0	1	10	1	0	1	1		

Let's move on to the next example. We are given the function:

$$F(A,B,C,D,E) = \prod M(4, 6, 12, 14, 20, 21, 22, 23)$$

Again, recall that for product of maxterms, the output for the corresponding maxterms are 0, while other minterms will have an output of 1.

For a **5-variable K-map**, we treat it as **two 4-variable K-maps**, one for **A = 0** and one for **A = 1**.

A = 0

Q0 | Q4 | Q12 | Q8

Q1 | Q5 | Q13 | Q9

Q3 | Q7 | Q15 | Q11

Q2 | Q6 | Q14 | Q10

A = 1

Q16 | Q20 | Q28 | Q24

Q17 | Q21 | Q29 | Q25

Q19 | Q23 | Q31 | Q27

Q18 | Q22 | Q30 | Q26

To complete the K-map, we **place 0s in the cells corresponding to the given maxterms**, and all other cells are filled with 1s. Hence, we will have:

A = 0

1 | 0 | 0 | 1

1 | 1 | 1 | 1

1 | 1 | 1 | 1

1 | 0 | 0 | 1

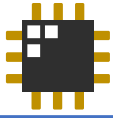
A = 1

1 | 0 | 1 | 1

1 | 0 | 1 | 1

1 | 0 | 1 | 1

1 | 0 | 1 | 1



Forming Boolean expressions from K-maps

- We have just translated a truth table into a K-map
 - K-map: a grid with logically adjacent cells
- Having logically adjacent cells allows us to easily find 'patterns' to give us the simplified Boolean expressions already!

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

B'

Looking at the truth table is not intuitive
when creating Boolean expressions...

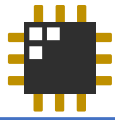
What does this look like in a K-map?

34

With a **Karnaugh Map (K-map)**, simplifying Boolean expressions becomes much more straightforward because it visually groups terms that can be combined. Instead of relying on algebraic manipulation, which can be tedious and error-prone, the K-map helps us **quickly identify patterns** that lead to simplified expressions.

In contrast, using a **truth table** alone does not provide an intuitive way to simplify Boolean functions. While a truth table shows all possible input-output combinations, it does not inherently reveal which terms can be grouped or minimized. Without a structured approach like the K-map, one would have to manually apply Boolean algebra rules, which can be complex and time-consuming.

Now, take the given truth table and represent it using a K-map.



Forming Boolean expressions from K-maps

$$F(A,B,C) = \sum m(0, 1, 4, 5)$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

		AB			
		00	01	11	10
C	0	1	0	0	1
	1	1	0	0	1

35

Here, we start by identifying which rows in the truth table have an output of 1, beginning from row 0.

From the truth table, the function includes rows 0, 1, 4, and 5. In terms of minterms, we can express this as:

$$F(A,B,C) = \sum m(0, 1, 4, 5)$$

Now that we have identified the minterms, we simply place them in the K-map according to the designated positions:

K-map layout:

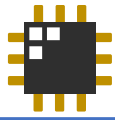
Q0 | Q2 | Q6 | Q4

Q1 | Q3 | Q7 | Q5

After filling in the corresponding outputs, we get:

1 | 0 | 0 | 1

1 | 0 | 0 | 1



Forming Boolean expressions from K-maps

Physically adjacent:
They differ in only one bit

Logically adjacent:
They differ in only one
variable (C)

AB		00	01	11	10
C	0	1	0	0	1
	1	1	0	0	1

$$A'B'C' + A'B'C = A'B'(C' + C) = A'B'$$

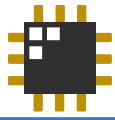
36

Now, let's analyze the K-map. The binary representation of the given terms is:

• $A'B'C' \rightarrow 000$ (orange)

• $A'B'C \rightarrow 001$ (green)

When we group these two adjacent cells together (highlighted in red), we notice that they share the same $A'B'$ part, but differ only in C . Thus, the simplified Boolean expression is $A'B'$.



Forming Boolean expressions from K-maps

Physically adjacent:
They differ in only one bit

Logically adjacent:
They differ in only one
variable (C)

		AB			
		00	01	11	10
C	0	1	0	0	1
	1	1	0	0	1

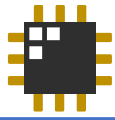
$$AB'C' + AB'C = AB'(C' + C) = AB'$$

37

Now, moving to the other side, let's first determine the binary representations:

- $AB'C' \rightarrow 100$ (blue)
- $AB'C \rightarrow 101$ (yellow)

By grouping these two adjacent cells, we observe that $A = 1$ and $B = 0$ remain constant in both cases, while C varies (C' and C). Thus, the simplified Boolean expression is AB' .



Forming Boolean expressions from K-maps

Physically adjacent:
They differ in only one bit

Logically adjacent:
They differ in only one
variable (A)

		AB			
C		00	01	11	10
	0	1	0	0	1
	1	1	0	0	1

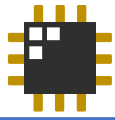
$$A'B' + AB' = B'(A' + A) = B'$$

38

As mentioned earlier, the first and last columns/rows in a K-map are logically adjacent, even though they appear on opposite sides. This means we can group the columns **00** and **10** together.

Looking at the red group, the only common term is $B = 0$.

Thus, the final simplified Boolean expression for this K-map is B' .



Forming Boolean expressions from K-maps

The **equivalent expression** of a group is **simply the common term(s) covered by the group**.

Recall that the last column is logically adjacent (1-bit change) to the first column!

		AB			
C		00	01	11	10
	0	1	0	0	1
	1	1	0	0	1

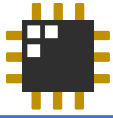
The equivalent Boolean expression for this grouping: **B'**

39

As mentioned earlier, the first and last columns/rows in a K-map are logically adjacent, even though they appear on opposite sides. This means even though they seem separated visually, they can still be grouped together when simplifying the Boolean expression.

For this example, we can group the columns **00** and **10** together. To determine the simplified expression, we identify the common term (unchanged variable) within this group. Here, $B = 0$ stays the same. Thus, the final simplified Boolean expression for this K-map is **B'** .

To make this process easier, the equivalent expression of a group is simply the common term(s) covered by the group.



Reading K-maps

- Group only entries that are **logically adjacent** (1-bit change)
- Groups should be of size **$2^n \times 2^m$** , where n and m are integers
 - Example: 1x1, 1x2, 1x4, 2x4, 1x8, 2x2, 4x4, 2x8, etc.
 - Not valid groups: 1x6, 2x5, 3x1, etc.
- Always **aim for the largest groupings possible!**
- **Groups may overlap** as long as each group contains at least one unique entry not covered by any other group.
 - **No redundant groups!** It is redundant when all of its entries are already covered by another group

40

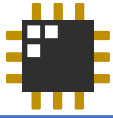
When grouping cells in a Karnaugh Map (K-map), it's essential to follow specific rules for proper simplification.

First, groups must be **logically adjacent**, meaning they should differ by only **one bit** in their binary representation.

Second, the size of each group must be a power of **2** (e.g., 2, 4, 8, etc.), ensuring that groups follow valid patterns such as **1x1, 1x2, 2x2, 1x4**, and so on.

Larger groups lead to simpler Boolean expressions, so it's best to aim for the **biggest possible groupings** while covering as many ones as possible.

Additionally, groups **may overlap** as long as each group contains at least **one unique cell** not covered by any other group. However, redundant groups that do not contribute to further simplification should be avoided.



Example 1: Reading the K-map

Find the minimized **SOP** expression given the K-map

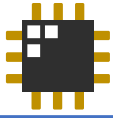
C \ AB	AB			
	00	01	11	10
0	0	1	0	0
1	1	1	1	1

41

Since the entire second row is filled with 1s, it simplifies to just **C**.

Additionally, the second column is also completely filled with 1s, which simplifies to **A'B**.

Therefore, the minimized **Sum of Products (SOP)** expression is **A'B + C**.



Example 1: Reading the K-map

Find the minimized **SOP** expression given the K-map

AB		00	01	11	10
C	0	0	1	0	0
	1	1	1	1	1

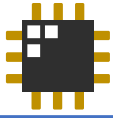
$A'B + C$

42

Since the entire second row is filled with 1s, it simplifies to just **C**.

Additionally, the second column is also completely filled with 1s, which simplifies to **A'B**.

Therefore, the minimized **Sum of Products (SOP)** expression is **$A'B + C$** .



Example 2: Reading the K-map

Find the minimized **POS** expression given the K-map

C \ AB	AB			
	00	01	11	10
0	0	1	0	0
1	1	1	1	1

43

Again, as a reminder, to obtain the minimized **Product of Sums (POS)** expression, we focus on grouping the **cells with an output of 0**. When writing the POS expression, recall that:

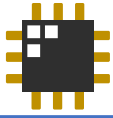
- A **non-primed** variable (e.g., A) represents **0** in the binary form.
- A **primed** variable (e.g., A') represents **1** in the binary form.

In this case, the **0s** are located at **cells 110, 100, and 000**.

First, consider **cells 110 and 100**—here, **A** and **C'** remain unchanged, so they form the term **(A' + C)**.

Next, look at **cells 100 and 000**—in this case, **B'** and **C'** remain unchanged, forming the term **(B + C)**.

Thus, the final minimized **POS expression** is **(A' + C)(B + C)**.



Example 2: Reading the K-map

Find the minimized **POS** expression given the K-map

AB		00	01	11	10
C	0	0	1	0	0
	1	1	1	1	1

$$(A' + C)(B + C)$$

44

Again, as a reminder, to obtain the minimized **Product of Sums (POS)** expression, we focus on grouping the **cells with an output of 0**. When writing the POS expression, recall that:

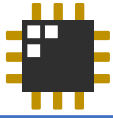
- A **non-primed** variable (e.g., A) represents **0** in the binary form.
- A **primed** variable (e.g., A') represents **1** in the binary form.

In this case, the **0s** are located at **cells 110, 100, and 000**.

First, consider **cells 110 and 100**—here, **A** and **C'** remain unchanged, so they form the term **(A' + C)**.

Next, look at **cells 100 and 000**—in this case, **B'** and **C'** remain unchanged, forming the term **(B + C)**.

Thus, the final minimized **POS expression** is **(A' + C)(B + C)**.



Example 3: Reading the K-map

Find the minimized **SOP** expression given the K-map

CD \ AB	AB			
	00	01	11	10
00	1	0	1	1
01	0	0	1	0
11	0	1	0	0
10	1	0	1	1

45

First, grouping the first and fourth rows with the third and fourth columns gives the expression AD' .

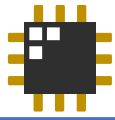
Second, grouping the third column with the first and second rows results in ABC' .

Third, for row 3, column 2, since there are no 1s in its adjacent cells, the expression is $A'BCD$.

Lastly, the four corner cells form a group, simplifying to $B'D'$.

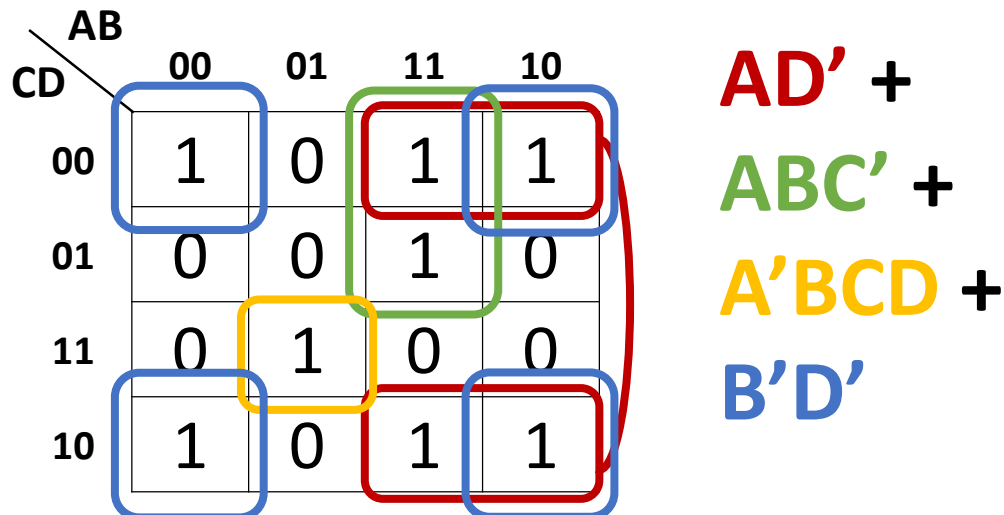
By ORing all these terms together, we get:

$$AD' + ABC' + A'BCD + B'D'$$



Example 3: Reading the K-map

Find the minimized SOP expression given the K-map



46

First, grouping the first and fourth rows with the third and fourth columns gives the expression AD' .

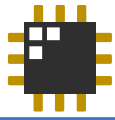
Second, grouping the third column with the first and second rows results in ABC' .

Third, for row 3, column 2, since there are no 1s in its adjacent cells, the expression is $A'BCD$.

Lastly, the four corner cells form a group, simplifying to $B'D'$.

By ORing all these terms together, we get:

$$AD' + ABC' + A'BCD + B'D'$$



Example 4: Reading the K-map

Find the minimized **POS** expression given the K-map

CD \ AB				
	00	01	11	10
00	1	0	1	1
01	0	0	1	0
11	0	1	0	0
10	1	0	1	1

47

For the **Product of Sums (POS)** expression, we focus on grouping the cells that output **0**. When writing the POS expression, recall that:

- A **non-primed** variable (e.g., A) represents **0** in the binary form.
- A **primed** variable (e.g., A') represents **1** in the binary form.

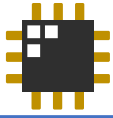
First, grouping the **first and fourth columns** with the **second and third rows** results in **$(B + D')$** .

Second, grouping the **third and fourth columns** along the **third row** gives us **$(A' + C' + D')$** .

Third, grouping the **first and fourth rows** along the **second column** simplifies to **$(A + B' + D)$** .

Lastly, grouping the **first and second columns** along the **second row** results in **$(A + C + D')$** .

By ANDing all these terms together, the final **POS expression** is:
 $(B + D')(A' + C' + D')(A + B' + D)(A + C + D')$.



Example 4: Reading the K-map

Find the minimized **POS** expression given the K-map

AB					
CD	00	01	11	10	
	00	01	11	10	
00	1	0	1	1	
01	0	0	1	0	
11	0	1	0	0	
10	1	0	1	1	

$$(B + D')$$

$$(A' + C' + D')$$

$$(A + B' + D)$$

$$(A + C + D')$$

48

For the **Product of Sums (POS)** expression, we focus on grouping the cells that output **0**. When writing the POS expression, recall that:

- A **non-primed** variable (e.g., A) represents **0** in the binary form.
- A **primed** variable (e.g., A') represents **1** in the binary form.

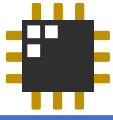
First, grouping the **first and fourth columns** with the **second and third rows** results in **$(B + D')$** .

Second, grouping the **third and fourth columns** along the **third row** gives us **$(A' + C' + D')$** .

Third, grouping the **first and fourth rows** along the **second column** simplifies to **$(A + B' + D)$** .

Lastly, grouping the **first and second columns** along the **second row** results in **$(A + C + D')$** .

By ANDing all these terms together, the final **POS expression** is:
 $(B + D')(A' + C' + D')(A + B' + D)(A + C + D')$.

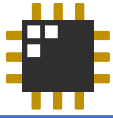


Incompletely Specified Functions

- There are cases where the ***output does not matter*** for certain input combinations.
 - Some specific combination(s) of input values cannot occur
 - The input can happen but the output is ignored
- Output values that are not significant are treated as **don't cares**
 - Usually represented as 'x' or 'd' in the truth table
- We can exploit don't cares during logic minimization!

49

In some cases, certain input combinations either never occur or their corresponding output is irrelevant. These scenarios introduce **don't care conditions**, which are represented as '**x**' or '**d**' in a truth table. Since these values do not impact the circuit's operation, they provide flexibility during **logic minimization**.



K-maps and don't cares

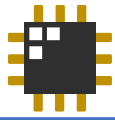
$$F(A,B,C) = \sum m(1, 5, 7) + d(3)$$

C \ AB				
	00	01	11	10
0	0	0	0	0
1	1	X	1	1

When writing minimized expressions, decide if treating the don't care as 0 or 1 would help simplify the circuit

50

By strategically treating **don't care values** as either **0** or **1**, we can form **larger groups** in a **K-map**, leading to a **simplified Boolean expression** with fewer terms and logic gates. This optimization helps create more efficient digital circuits.



K-maps and don't cares

C \ AB	00	01	11	10
	0	0	0	0
1	1	0	1	1

Treating the don't care as a '0' results in these groupings

C \ AB	00	01	11	10
	0	0	0	0
1	1	1	1	1

Treating the don't care as a '1' results in this group

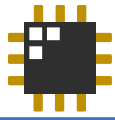
51

Here's an example of how don't care conditions can help in minimization.

Looking at the **K-map on the left**, if we treat the don't care as **0**, we end up with two separate groupings, which is not ideal. Since our goal is to **minimize the number of groups**, we should take advantage of the don't care condition.

Now, in the **K-map on the right**, we assign **1** to the don't care. This allows us to form **one large group instead of two**, leading to a more simplified expression.

This demonstrates why strategically using don't care values can help achieve **better logic minimization**.



Example 1: K-map with don't cares

Find the minimized **SOP** expression given the K-map

CD \ AB	AB			
	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

52

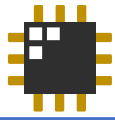
Here's another example of how don't care conditions can be used in a K-map.

First, by treating the don't care as **1** in the second and third rows along the third column, we can form a **larger group** in that column. This allows us to simplify the expression to **AB**.

Next, we group the **corner cells**, which results in the term **B'D'**.

ORing them together we'll get:

$$\mathbf{AB + B'D'}$$



Example 1: K-map with don't cares

Find the minimized **SOP** expression given the K-map

AB		00	01	11	10
CD	00	1	0	1	1
	01	0	0	x	0
	11	0	x	x	0
	10	1	x	1	1

$$AB + B'D'$$

53

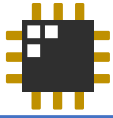
Here's another example of how don't care conditions can be used in a K-map.

First, by treating the don't care as **1** in the second and third rows along the third column, we can form a **larger group** in that column. This allows us to simplify the expression to **AB**.

Next, we group the **corner cells**, which results in the term **B'D'**.

ORing them together we'll get:

$$AB + B'D'$$



Example 2: K-map with don't cares

Find the minimized **POS** expression given the K-map

CD \ AB	AB			
	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

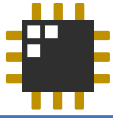
54

First, we can make the don't cares along the second and third row to 0, to form a group of 8 cells, resulting to D' .

Second, we can make the don't cares 0 along the second column, so we can group the entire second row and result to $(A+B')$.

ANDing them together, we'll get:

$$(D')(A+B')$$



Example 2: K-map with don't cares

Find the minimized **POS** expression given the K-map

CD \ AB				
	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

$$(D') (A+B')$$

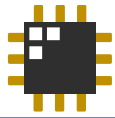
55

First, we can make the don't cares along the second and third row to 0, to form a group of 8 cells, resulting to D' .

Second, we can make the don't cares 0 along the second column, so we can group the entire second row and result to $(A+B')$.

ANDing them together, we'll get:

$$(D')(A+B')$$



K-map with don't cares

Same truth table (same K-map), but different expressions?

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

$AB + B'D'$

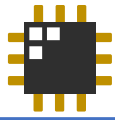
AB \ CD	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

$(D')(A+B')$

56

The same truth table or K-map can result in different **Sum of Products (SOP)** and **Product of Sums (POS)** expressions, especially when **don't care conditions** are present.

This happens because **don't cares** can be assigned as either **1** or **0**, depending on which grouping leads to a simpler expression. Since **SOP groups 1s** while **POS groups 0s**, the way don't cares are utilized can lead to different but equally valid expressions.



K-map with don't cares

Same truth table (same K-map), but different expressions?

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

$$AB + B'D'$$

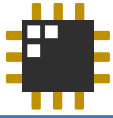
AB \ CD	00	01	11	10
00	1	0	1	1
01	0	0	x	0
11	0	x	x	0
10	1	x	1	1

$$(D')(A+B')$$

Since some of the don't cares were treated differently during the grouping process, the simplified SOP and POS expressions might not end up being logically equivalent (but this is fine)

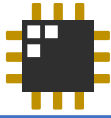
57

Ultimately, the key is to strategically assign don't cares in a way that **minimizes the logic function** while ensuring the desired form, SOP or POS, is achieved.



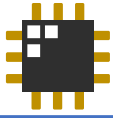
Playing around with K-maps!

AB		00	01	11	10
C	0				
	1				



Playing around with K-maps!

CD \ AB				
	00	01	11	10
00				
01				
11				
10				



Playing around with K-maps!

		A = 0			
DE	BC	00	01	11	10
00					
01					
11					
10					

		A = 1			
DE	BC	00	01	11	10
00					
01					
11					
10					